

b) Investigación TESTING

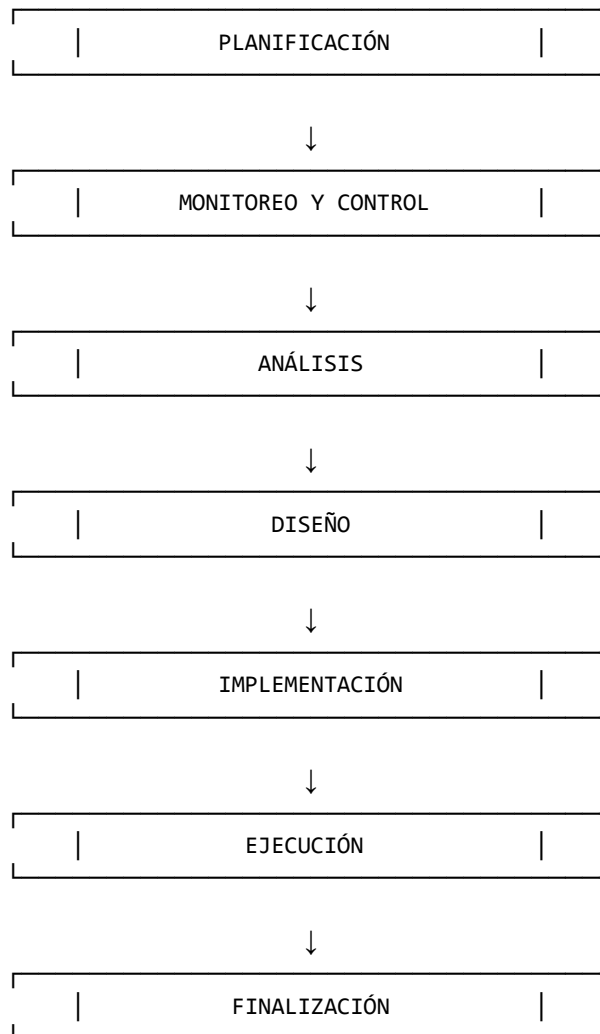
1. Plan de Pruebas

Un Plan de Pruebas de Software es un documento que define qué se va a probar, cómo, cuándo, con qué recursos y cuáles serán los criterios para determinar si las pruebas fueron satisfactorias. Permite organizar el proceso de testing y controlar sus resultados.

Generalmente incluye: objetivos, alcance, recursos, responsables, estrategia de pruebas, ambiente, datos de prueba, cronograma, criterios de entrada y salida, riesgos y métricas.

Ciclo de vida de las pruebas

El ciclo de vida comprende las actividades de planificación, monitoreo y control, análisis, diseño, implementación, ejecución y finalización de las pruebas. En metodologías iterativas estas actividades pueden repetirse en cada iteración.



2. Tipos de Prueba

Tipo	¿Qué verifica?	Objetivo
------	----------------	----------

Unitaria	Una unidad o componente.	Comprobar que funciona correctamente de forma aislada.
Integración	Interacción entre componentes.	Detectar problemas de comunicación entre módulos.
Sistema	El sistema completo.	Verificar el cumplimiento de los requisitos.
Aceptación	El sistema desde la perspectiva del usuario.	Comprobar que satisface las necesidades definidas.
Funcional	Las funciones del sistema.	Verificar que cada funcionalidad produce el resultado esperado.
Regresión	Funcionalidades después de cambios.	Detectar errores introducidos por modificaciones.
Rendimiento	Respuesta y comportamiento.	Medir el comportamiento bajo diferentes condiciones.
Carga	Una carga determinada.	Evaluar el sistema con la cantidad de usuarios/solicitudes prevista.
Estrés	Carga superior a la prevista.	Observar el comportamiento ante sobrecarga.
Seguridad	Protección y acceso a datos.	Detectar vulnerabilidades y accesos no autorizados.
Usabilidad	Facilidad de uso.	Evaluar si el sistema es comprensible y fácil de utilizar.

3. Patrones de Automatización de Pruebas

POM (Page Object Model): Representa cada página o componente de una aplicación mediante objetos/clases. Separa la lógica de las pruebas de los detalles de la interfaz y facilita la reutilización y el mantenimiento.

Pruebas basadas en datos: Separan los datos de prueba de la lógica de ejecución. Una misma prueba puede ejecutarse con distintos usuarios, contraseñas o valores, evitando repetir código.

BDD (Behavior-Driven Development): Describe el comportamiento esperado mediante ejemplos comprensibles para técnicos y personas del negocio. Una estructura habitual es Given–When–Then (Dado–Cuando–Entonces).

Pruebas de carga: Evalúan el comportamiento del sistema cuando recibe una carga determinada, por ejemplo, una cantidad esperada de usuarios simultáneos. Se pueden medir tiempos de respuesta, errores y recursos utilizados.

Pruebas de estrés: Someten al sistema a cargas superiores a las previstas para analizar cómo responde ante situaciones extremas y conocer sus límites y capacidad de recuperación.

Resumen

El testing permite comprobar la calidad del software mediante un proceso planificado. El plan de pruebas organiza las actividades, los tipos de prueba permiten evaluar distintos aspectos del sistema y los patrones de automatización facilitan crear pruebas reutilizables y mantenibles.